Smart Food Divider Cabinet System

Group 3

賴琮運

潘怡璇

蕭宜芳

1. Introduction

People usually buy large packages of groceries and share them with each other. In our lab, it is difficult and time-consuming for people to share the food and split the money precisely. This often leads to confusion and discrepancies in how costs are divided. For example, when someone takes a portion of a shared item like a bag of Oreos, it can be challenging to track who took what and how much they owe.

People are looking for a system that simplifies this entire process, ensuring fairness and efficiency. A system that can automatically track the retrieval of shared items and instantly calculate and communicate the corresponding charges would be highly beneficial. This would eliminate the need for manual tracking and calculation, reducing potential disputes and saving time. Essentially, such a system would allow for real-time billing and accountability, making it easier for everyone to share and enjoy communal groceries without worrying about the complexities of splitting the costs.

2. Design Consideration

This section introduces the hardware, software, and architecture of this project.

2.1. Software and Hardware

Our device is divided into software and hardware parts. (See Figure1.) For the software database, we use Google Sheets, where we perform the calculations for the billing information. The front-end user interface is managed through a Line Bot, which creates a group for billing and operates the entire device. The hardware components include the Raspberry Pi,

Electronic Lock, and Force Sensor. The Raspberry Pi acts as the single-board computer and the core of the device. The Electronic Lock controls the opening and closing of the food container, while the Force Sensor detects pressure changes.

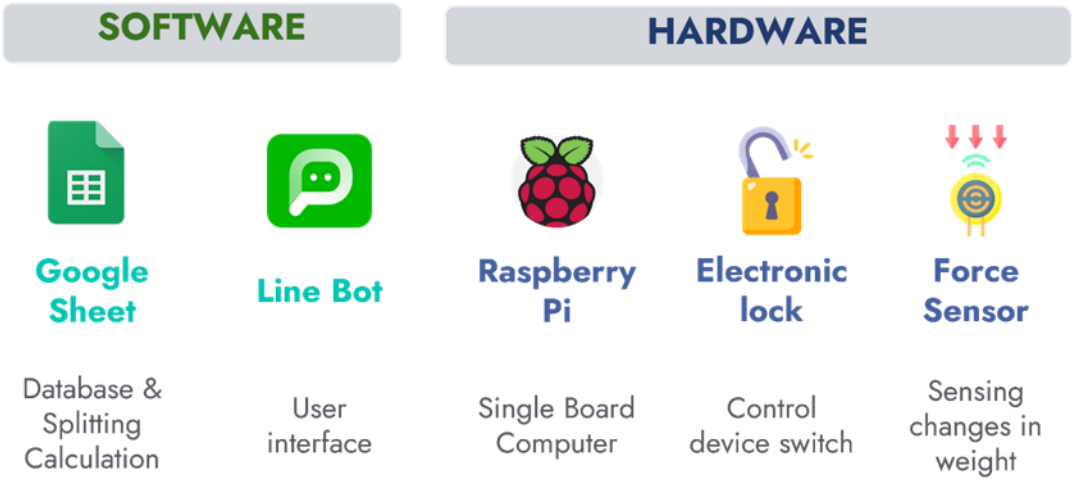


Figure2. the software and hardware in our smart food divider cabinet system

2.2. Device Design

This is our Smart Food Divider Cabinet Device (See Figure2.), which is composed of four parts: the Cabinet Body, Raspberry Pi, Magnetic Lock Module, and Force Sensor Module. The Cabinet Body is a transparent plastic container used to store the food and items to be divided. The Raspberry Pi is the core of the system, controlling the Magnetic Lock Module and reading the weight changes from the Force Sensor Module. It also connects the device to a Line bot and Google Sheets. The Magnetic Lock Module serves as the lock for the food container, allowing users to control the opening and closing of the cabinet through the Line bot. Finally, the Force Sensor Module is the key component for achieving smart billing

functionality by measuring changes in weigh.

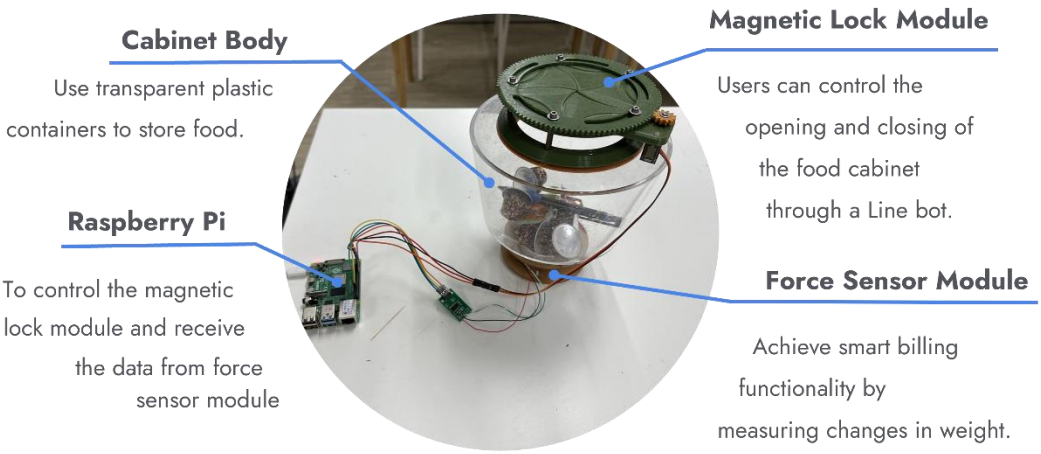


Figure2. our Smart Food Divider Cabinet Device

2.3. System Flowchart

This is our system flowchart. (See Figure3.) The LINE BOT is the main control interface, allowing users to send commands. The LINE BOT sends open/close messages to the Raspberry Pi. Upon receiving these messages, the Raspberry Pi sends commands to the Electronic Lock to open or close the food container. Additionally, the Force Sensor sends pressure change data to the Raspberry Pi, which then forwards it to the backend database, Google Sheets, for storage. The LINE BOT also transmits the billing information entered by the users to Google Sheets. Finally, the calculated billing data is retrieved from Google Sheets and sent back to the LINE BOT, informing the users of how much money they owe to whom.

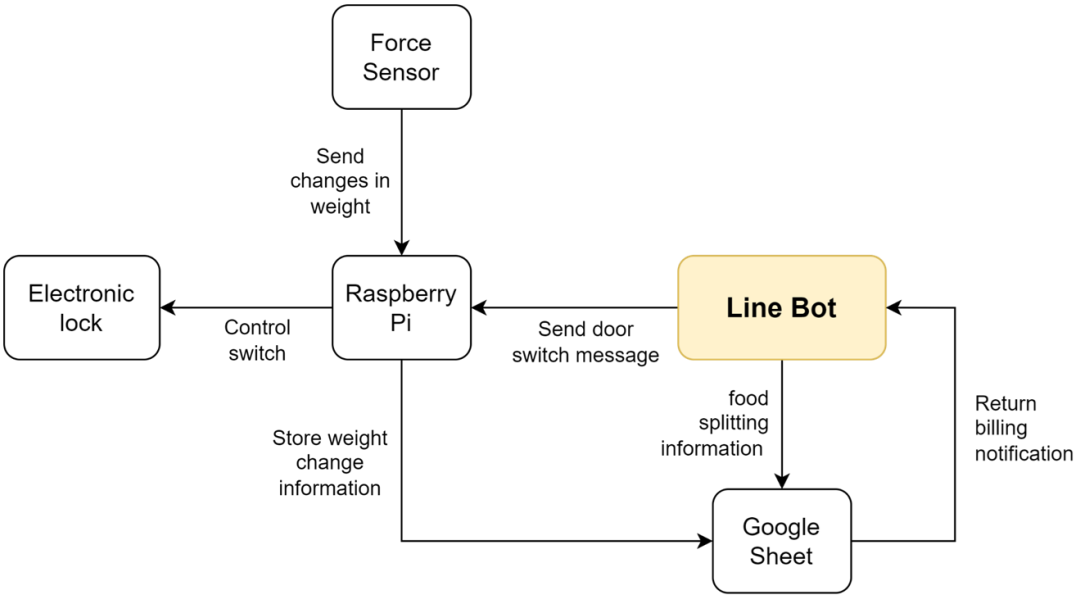


Figure3. system flowchart

3. Device Functions

Our device has the following three functions.

I. Secure Access Control

By controlling the food cabinet's switch through a Line bot, individuals not involved in the billing process cannot open the food cabinet.

II. Theft Prevention Mechanism

If someone tries to take food while it’s being placed, the food cabinet switch will immediately close (see figure4.) and continuously send back the word 'greedy'.(see figure5.)



Figure4. theft prevention mechanism – Immediately close the switch



Figure5. theft prevention mechanism – continuously send back 'greedy'

III. Smart Billing Functionality

After the user takes the food, the Line bot will send back billing information, indicating to whom the money for this food retrieval should be given.

4. Implementation

4.1. Front-end user interface - Line bot

I. placing food

When the user inputs **“placing food [payer] [food name] [quantity] [total price]”** into the Line bot group, the food container's switch will open. At this point, the user can place the food inside. Once the placement is complete and the food container no longer detects any weight change, the switch will automatically close. At the same time, the Line bot will also send back a **“Success!”** message.

II. taking food

Participants in the billing process can input **“taking food”** into the Line bot to open the food container's switch and retrieve food from the container. At this point, the smart billing device will detect the weight change and compare it with the backend database to determine what food the user has taken and calculate how much money the user owes and to whom.

After the user takes the food and the container no longer detects any weight change, the container will automatically close and the Line bot will send back a **“Complete”** message.

III. Billing

After the user has taken the food, they can input the message **“billing”** to find out the billing information. The Line bot will then send back the message **“How much money do you need to give to whom”**



Figure6. Operation Process: Placing Food, Taking Food, Billing

4.2. Back-end database - Google sheet

We use Google Sheets to establish the backend database for our device, which can instantly store weight information and calculate weight changes to determine the current status of food placement in the container. The Line bot can then read the information from this database to provide users with billing details, informing them of how much money they owe to whom.

	A	B	C
1	Time	Total_Weight	Button_Type
2	2024-05-27 22:12:41	-0.1483679525	put
3	2024-05-27 22:12:46	101.5248928	put
4	2024-05-27 22:12:52	101.5001649	put
5	2024-05-27 22:14:54	0.08700589808	put
6	2024-05-27 22:15:00	101.4305601	put
7	2024-05-27 22:15:06	101.2492215	put
8	2024-05-28 20:06:45	0.1318826245	put
9	2024-05-28 20:06:51	101.4836795	put
10	2024-05-28 20:06:57	99.75272008	put
11	2024-05-28 20:07:02	99.01912298	put
12	2024-05-28 20:21:29	-0.09616441367	put
13	2024-05-28 20:21:35	91.59523025	put
14	2024-05-28 20:21:41	83.68227278	put
15	2024-05-28 20:21:46	83.32783822	put
16	2024-05-28 20:38:10	-0.09250100744	put
17	2024-05-28 20:38:13	101.7785837	put
18	2024-05-28 20:38:17	101.6631864	put
19	2024-05-28 20:38:21	101.6302158	put
20	2024-05-28 21:01:55	0.07967908561	put
21	2024-05-28 21:01:59	101.3902627	put
22	2024-05-28 21:02:03	101.7529399	put
23	2024-05-28 21:02:06	101.4479613	put
24	2024-05-28 21:02:10	101.5551159	put

H8

	A	B	C	D	E
1	Time	User	Food	Portion	Price
2	2024-05-28 12:15:46	A	辣味麥克雞塊	5	
3	2024-05-28 20:20:39	A	辣味麥克雞塊	5	
4	2024-05-28 20:31:25	A	辣味麥克雞塊	5	99
5	2024-05-28 20:52:41	A	辣味麥克雞塊	5	99
6	2024-05-28 21:01:04	A	辣味麥克雞塊	5	99
7	2024-05-29 21:01:04	B	nescafe	2	130
8	2024-05-28 21:01:04	A	辣味麥克雞塊	5	99
9	2024-05-29 21:01:04	C	donut	3	120
10	2024-05-29 21:01:04	B	nescafe	2	68
11	2024-05-28 21:01:04	A	辣味麥克雞塊	2	99
12	2024-05-28 21:01:04	A	辣味麥克雞塊	7	134
13	2024-05-30 20:31:30	Reg	糖醋醬	2	60
14	2024-05-30 20:31:30	Reg	Nescafe	3	80
15	2024-05-30 20:31:30	Reg	Cover	1	50
16	2024-06-04 14:50:02	A	辣味麥克雞塊	5	99
17	2024-06-04 14:52:51	A	辣味麥克雞塊	5	99
18	2024-06-04 15:08:09	A	辣味麥克雞塊	5	99
19	2024-06-04 19:07:19	A	辣味麥克雞塊	5	99
20	2024-06-04 19:12:28	A	咖啡	5	99

	A	B	C	D	E	F	G	H
1	Index	User	Food	Portion	Total_Weight	Unit_Weight	Total_Price	Unit_Price
2	1	Reg	Test	0	9999	9999	0	30
3	2	A	糖醋醬	2	202.8015899	101.400795	40	20
4	3	A	球	5	29.75587061	7	25	5
5	4	A	咖啡膠囊	7	153.1554521	21.87935029	210	30

Figure7. The data storage and computation process in the backend database

4.3. Force sensor

The weight change values measured by the pressure sensor. (See figure8.)

```
put
202.1037110305164
209.34077004799084
209.25010074367168
209.5385939846872
43.198.152.120 - - [05/Jun/2024 13:53:28] "POST /toggle_motor HTTP/1.1" 200 -
toggle_motor
takeclockwise
take
234.48089533648408
201.6915778290657
201.71630582115273
201.71630582115273
43.198.152.120 - - [05/Jun/2024 13:54:26] "POST /toggle_motor HTTP/1.1" 200 -
toggle_motor
takeclockwise
take
796.589368795106
-1.1109279407991421
-1.1356559328861846
-1.0202586364799862
43.198.152.120 - - [05/Jun/2024 13:56:41] "POST /toggle_motor HTTP/1.1" 200 -
toggle_motor
putclockwise
put
40.753562662563866
201.14756200315077
200.90028208228034
201.19701798732484
43.198.152.120 - - [05/Jun/2024 13:57:35] "POST /toggle_motor HTTP/1.1" 200 -
toggle_motor
putclockwise
put
200.34802359233638
241.01732791149234
241.25636516833373
241.42946111294305
43.198.152.120 - - [05/Jun/2024 13:59:38] "POST /toggle_motor HTTP/1.1" 200 -
toggle_motor
putclockwise
put
249.16107997215835
328.6780598600581
371.8531340440344
415.1271201963588
415.1106348683008
415.2837308129101
43.198.152.120 - - [05/Jun/2024 14:02:54] "POST /toggle_motor HTTP/1.1" 200 -
```

Figure8. Weight Change Values Measured by Pressure Sensor